

Problema de Afetação

Neste tipo de problema, pretende-se afetar (atribuir, distribuir), da forma mais económica, indivíduos a tarefas, sendo que a palavra “indivíduos” se refere a entidades que não têm que ser pessoas, podendo ser máquinas, veículos, fábricas, ou até mesmo *slots* de tempo.

Um exemplo típico de um problema de afetação é o seguinte:

Um documento escrito em Português precisa de ser traduzido para Chinês, Japonês, Alemão e Inglês (línguas denotadas por C, J, A, I, respetivamente), sendo 4 pessoas designadas para a sua tradução. Embora essas pessoas sejam capazes de traduzir para os 4 idiomas, a ideia é ter apenas uma pessoa a fazer a tradução para cada um dos idiomas. A tabela que se segue, mostra o tempo (em horas) que cada pessoa demora a traduzir o documento para os vários idiomas. O objetivo é minimizar o tempo total utilizado na tradução.

		Tarefa			
		C	J	A	I
Pessoas	1	2	15	13	4
	2	10	4	14	15
	3	9	14	16	13
	4	7	8	11	9

Um problema de afetação deve satisfazer os seguintes pressupostos:

- O número de indivíduos deve coincidir com o número de tarefas (esse número será denotado por n)
- Cada indivíduo será afeto a, exatamente, uma tarefa
- Cada tarefa será executada por, exatamente, um indivíduo
- Há um custo c_{ij} associado à realização da tarefa j ($j = 1, 2, \dots, n$) pelo indivíduo i ($i = 1, 2, \dots, n$)
- O objetivo é determinar como é que devem ser feitas as afetações dos n indivíduos às n tarefas, de forma a minimizar o custo total.

O modelo matemático de um problema de afetação usa as seguintes variáveis de decisão:

$$x_{ij} = \begin{cases} 1 & \text{se o indivíduo } i \text{ realizar a tarefa } j \\ 0 & \text{caso contrário} \end{cases}$$

para $i = 1, 2, \dots, n$ e $j = 1, 2, \dots, n$.

Estas variáveis x_{ij} designam-se por binárias, uma vez que só podem assumir valor 0 ou 1. Normalmente usam-se para representar decisões do tipo *Sim/Não*, sendo que neste caso as decisões são: “*Deve o indivíduo i realizar a tarefa j ?*”

Considerando z o custo total, a formalização matemática de um problema de afetação será:

$$\text{Minimizar } z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

sujeito a,

$$\sum_{j=1}^n x_{ij} = 1 \quad \text{para } i = 1, 2, \dots, n$$

$$\sum_{i=1}^n x_{ij} = 1 \quad \text{para } j = 1, 2, \dots, n$$

$$x_{ij} \geq 0, \quad \text{para } i = 1, 2, \dots, n \text{ e } j = 1, 2, \dots, n$$

$$(x_{ij} \text{ binária, para } i = 1, 2, \dots, n \text{ e } j = 1, 2, \dots, n)$$

Através do modelo matemático anterior, percebe-se que um problema de afetação pode ser considerado um problema de transportes, em que:

- os indivíduos são as ‘origens’ e as tarefas são os ‘destinos’;
- as disponibilidades em cada origem e as necessidades em cada destino são iguais a 1;
- o custo unitário de transporte é c_{ij} ;
- o valor de x_{ij} é inteiro e ≥ 0 .

A sua representação em termos de um quadro de transportes é a seguinte:

Tarefas Indivíduos	1	2	...	n	
1	c11 x11	c12 x12	...	c1n x1n	1
2	c21 x21	c22 x22	...	c2n x2n	1
...	...	...	...	...	...
n	cn1 xn1	cn2 xn2	...	cnn xnn	1
	1	1	...	1	

Qualquer afetação de um indivíduo a uma tarefa “esgota” a oferta e “satura” a procura respectivas. Assim, em qualquer SBA, existem apenas n variáveis básicas (células preenchidas), em vez das $(n + n - 1 = 2n - 1)$ necessárias; ou seja, qualquer SBA é “fortemente” degenerada, contendo $(n - 1)$ VB nulas, o que torna difícil a utilização dos métodos anteriormente aplicados na resolução dos problemas de transportes.

Em alternativa será usado o **método Húngaro**, desenvolvido e publicado em 1955 por Harold Kuhn, o qual permite resolver de forma rápida, problemas de afetação. A sua aplicação será explicada através do exemplo que se segue.

Exemplo

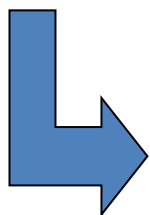
Numa fábrica foram instaladas **4 máquinas** e existem **4 tarefas** para executar. O objetivo da Direção da fábrica é estabelecer uma afetação máquina–tarefa recíproca e exclusiva, que corresponda a um tempo total de configuração mínimo. A tabela seguinte apresenta os tempos de configuração de cada máquina (em minutos) para cada tipo de tarefa.

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	14	5	8	7
Máquina 2	2	12	6	5
Máquina 3	7	8	3	9
Máquina 4	2	4	6	10

Aplicação do método Húngaro – Passo 1

(a) Encontrar o elemento mínimo em cada linha da matriz de custos. Formar uma nova matriz subtraindo esse custo em cada linha.

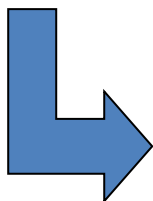
	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	14	5	8	7
Máquina 2	2	12	6	5
Máquina 3	7	8	3	9
Máquina 4	2	4	6	10



	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	9	0	3	2
Máquina 2	0	10	4	3
Máquina 3	4	5	0	6
Máquina 4	0	2	4	8

(b) Encontrar o custo mínimo em cada coluna da nova matriz e subtrair esse valor em cada coluna. Como resultado obtém-se a matriz de custos reduzidos.

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	9	0	3	2
Máquina 2	0	10	4	3
Máquina 3	4	5	0	6
Máquina 4	0	2	4	8



	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	9	0	3	0
Máquina 2	0	10	4	1
Máquina 3	4	5	0	4
Máquina 4	0	2	4	6

Aplicação do método Húngaro – Passo 2

Desenhar o número mínimo de linhas necessárias para cobrir todos os zeros na matriz de custos reduzidos. Se forem necessárias n linhas, então existirá uma solução ótima entre os zeros cobertos na matriz. Caso contrário, continuar para o Passo 3.

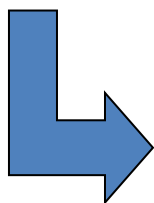
	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	9	0	3	0
Máquina 2	0	10	4	1
Máquina 3	4	5	0	4
Máquina 4	0	2	4	6

Como são necessárias 3 linhas ($< n = 4$) para cobrir todos os zeros, há que prosseguir para o Passo 3.

Aplicação do método Húngaro – Passo 3

Encontrar o menor elemento diferente de zero na matriz de custos reduzidos, não coberto pelas linhas (o qual será denotado por k). Subtrair k a cada elemento descoberto e adicionar k a cada elemento coberto por duas linhas. Voltar ao Passo 2.

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	9	0	3	0
Máquina 2	0	10	4	1
Máquina 3	4	5	0	4
Máquina 4	0	2	4	6



	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	10	0	3	0
Máquina 2	0	9	3	0
Máquina 3	5	5	0	4
Máquina 4	0	1	3	5

Aplicação do método Húngaro – Passo 2 (de novo)

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	10	0	3	0
Máquina 2	0	9	3	0
Máquina 3	5	5	0	4
Máquina 4	0	1	3	5

Como são necessárias 4 linhas ($n = 4$) para cobrir todos os zeros, o algoritmo termina!

Aplicação do método Húngaro – Solução final

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	10	0	3	0
Máquina 2	0	9	3	0
Máquina 3	5	5	0	4
Máquina 4	0	1	3	5

No final, é escolhido um conjunto de zeros de forma a que cada linha e cada coluna tenha apenas um zero selecionado.

Neste exemplo, nas linhas 3 e 4 existe apenas um zero, pelo que serão automaticamente selecionados. Essa escolha condicionará a da linha 2, que por sua vez condicionará a da linha 1.

Assim, a solução ótima do problema será: $x_{12} = 1$; $x_{24} = 1$; $x_{33} = 1$; $x_{41} = 1$; restantes $x_{ij} = 0$

Nota: Se for possível escolher arbitrariamente uma célula com zero, então haverá múltiplas soluções ótimas com o mesmo retorno total (z) para as afetações feitas. Nesse caso, caberá ao decisor final selecionar a solução mais adequada às suas necessidades.

O valor mínimo de z será obtido a partir da matriz de custos inicial.

	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4
Máquina 1	14	5	8	7
Máquina 2	2	12	6	5
Máquina 3	7	8	3	9
Máquina 4	2	4	6	10

Somando os custos assinalados (correspondentes aos zeros selecionados para a solução) obtém-se $z = 15$.

A conclusão final é que, a **máquina 1** deverá executar a **tarefa 2**, a **máquina 2** deverá executar a **tarefa 4**, a **máquina 3** deverá executar a **tarefa 3** e a **máquina 4** deverá executar a **tarefa 1**. Desta forma, o tempo total de configuração será mínimo e igual a **15** minutos.

Sugestão: Resolva o problema de tradução inicialmente apresentado (pág. V-51) pelo algoritmo Húngaro.